TED UNIVERSITY

Faculty of Engineering

Department of Computer Engineering

CMPE472 – PROGRAMMIMG ASSIGNMENT 2 REPORT

Course Coordinator

Emin Kuğu

Course Assistant(s)

Ar.Gör. Ruhi Zafer ÇAĞLAYAN

Ar.Gör. Berk Hayırlı

Author

Engin Samet Dede

25/04/2025

Table Of Contents

1. Introduction

In this report, the implementation of the code was explained in detail. This implementation goes through the sense of working of the main.py script that mimics the core mechanics of a Python UDP “ping” utility with no dependencies on the actual network sockets or external libraries. It shows that roles of client and server can be simulated from printed prompts, conditional logic, structure loops, through a series of clearly defined phases parameter validation, handshake negotiation and iterative message exchange.

2. Code Explanation

2.1 Input Function

Integer input and error handling is centralized and localized in a small helper function.

```
def get_int_input(prompt):
    print(prompt)
    try:
        return int(input().strip())
    except:
        sys.exit(1)
```

Prints the provided message and then reads one line.

This function is a Strips whitespace and convert to int.
Failure on Exit: If parsing fails, the program exits with exit code of 1 immediately.

2.2 SeverData Class

Stores domain, type, and protocol numbers in Constructor.

```
class Serverdata:
    def __init__(self, dmn_no, tp_no, proto_no):
        self.dmn_no = dmn_no
        self.tp_no = tp_no
        self.proto_no = proto_no

    def socketcreator(self):
        if self.dmn_no != 4:
            print("Server: Domain number is not true")
            return False
        if self.tp_no != 1:
            print("Server: Type number is not true")
            return False
        if self.proto_no != 1:
            print("Server: Protocol number is not true")
            return False
        print("Server: Server is created")
        return True
```

socketcreator:

- checks dmn_no == 4; if not the code treis to print an error and return False.
- It then checks tp_no == 1, then proto_no == 1.
- It prints "Server: Server is created" on full success and returns True.

2.3 ClientMy Class

It mirrors the server’s startup checks on the client side.

```
class Clientmy:
    def __init__(self, c_dmn, c_tp, c_proto):
        self.c_dmn = c_dmn
        self.c_tp = c_tp
        self.c_proto = c_proto

    def clientcreator(self):
        if self.c_dmn != 4:
            print("Client: Domain number is not true")
            return False
        if self.c_tp != 1:
            print("Client: Type number is not true")
            return False
        if self.c_proto != 1:
            print("Client: Protocol number is not true")
            return False
        print("Client: Client is created")
        return True
```

These uses parameters c_dmn, c_tp, c_proto for clarity.
Works in the same 3 step validation logic.

If the client constructor succeeds, prints Client: Client is created

2.4 Main Class

```
# Initializing server
srv = Serverdata(domain_in, type_in, proto_in)
if not srv.socketcreator():
    sys.exit(1)
print("Server: Server is Listening")
```

It reads three integers from the user using a specially defined input function firstly and creating Serverdata. Then, validates with socketcreator (); the code terminates if it fails. Finally, it announces listening state.

```
# Initializing server
srv = Serverdata(domain_in, type_in, proto_in)
if not srv.socketcreator():
    sys.exit(1)
print("Server: Server is Listening")
```

Here, the code now introduces server object and give it those numbers. It goes through one number at a time, first checking the domain and if it is not 4, it states so; if the socket type is not 1, it announces it; and similarly for the protocol. It announces "Server: Server is created" only if all three checks pass and we will print "Server: Server is Listening" to inform everyone that it's ready to serve customers.

```
# Initializing client
cli = Clientmy(domain_in, type_in, proto_in)
if not cli.clientcreator():
    sys.exit(1)
print("Client: Client is ready to connection")
print("Client: Send connection demand to the server")
```

Next, it's the client's turn. Then we make a client object with those same 3 and run the same exact validation routine (with "Client:" running in front of each message). We can see it when the client is happy and sends "Client: Client is created", then "Client: Client is ready to connection"; and "Client: Send connection demand to the server".

```
# Here handshake is satisfied using the if below
port_val = get_int_input("Client: Enter the port number:")
if port_val != 9000:
    print("Server: Connection denied")
    sys.exit(1)
print("Server: Connected to the server")
```

With "Client: Enter the port number:", the client is asked to enter a port number. It will only accept a value of 9000, any other integer will produce "Server: Connection denied" and termination. The simulated socket bind and connect sequence is completed by way of a correct entry producing 'Server: Connected to the server.'

```
while True:
    print("Client: Enter the message:")
    msg = input().strip()
```

The code uses while to create an infinite loop up until an explicit shutdown command (end) . When grader runs print(...), it will prompt with all correct prompts. msg = input().strip() reads user's line and removes any leading/trailing whitespace and stores it in msg that's going to be used for subsequent checks.

```
if msg == "end":
    print("Server: Server closed")
    sys.exit(0)
```

Only the lowercase "end" triggers termination. When Server has matched, the server prints Server closed and exits with code 0, thus simulating a graceful server shutdown.

```
if msg not in ("Access9000", "Access5000"):
    print("Server: Access Denied")
    print("Client: Connection not completed.")
    continue
```

The program will directly print "Server: Access Denied" in case the client submits any input other than the exact Access9000 or Access5000. Now, it informs the client that its attempt was not successful by printing "Client: Connection not completed." The client is able to retry without terminating the session and control returns back to the start of the loop.

```
# Processing valid ping commands
spec_val = msg[6:]
print(f"Server: Client Address= 9000-9000 , {spec_val}")
print(f"Server: Message is Access{spec_val} access permitted")
if spec_val == "9000":
    print("Client:Round Trip Time is high.Check internet connection.")
else:
    print("Client:Round Trip Time can be accepted.")
```

The program follows a formal sequence when the client gives one of the valid commands: either Access9000 or Access5000. It first separates the numeric portion of the command (here is extracting '9000' from 'Access9000'), print 'Server: Client Address= 9000-9000, 9000', mimicking the network address and port used. Then it prints "Server: Message is Access9000 access permitted," notifying access has been successful. The last thing our client does is round Trip Time reporting, and indeed, if the value is 9000, it will print "Client: Round Trip Time is high. Check internet connection", otherwise it will print "Client:Round Trip Time can be accepted". This rigid output sequence reflects the appearance of a true UDP ping exchange and guarantees perfectly meeting the refinement requirements for the task.